Engineering the Transition from Generative to Agentic AI

by Alexandru Bratosin

1. Main Thought

Enterprise AI is moving past isolated generative models toward autonomous, multi-agent systems. This transition shifts the focus from raw parameter count to system architecture, requiring robust data topologies, specialized memory stores, and deterministic execution frameworks. This paper outlines the technical requirements for deploying scalable, verifiable agentic workflows in production environments.

2. Foundational Architecture and Data Topologies

Selecting the right underlying topology dictates the ceiling of system performance. Modern enterprise workloads demand hybrid approaches.

Differentiating Predictive and Generative Modalities

- **Predictive Systems:** Optimized for deterministic boundaries. They utilize gradient boosting (e.g., XGBoost) and time-series forecasting models (e.g., ARIMA) to process high-velocity, structured numerical data.
- **Generative Systems:** Foundation models (LLMs) handle unstructured data synthesis. In production, these models are treated as reasoning engines rather than knowledge bases, decoupled from data storage to prevent hallucinated facts.

Graph Neural Networks (GNNs) for Relational Data

Standard transformer architectures struggle with highly connected enterprise data (e.g., supply chain logistics, network topologies). GNNs resolve this via message passing, where a node updates its state by aggregating features from its neighbors.

Architectures like GraphSAGE allow this aggregation to scale efficiently across massive, dynamic enterprise graphs without requiring the entire graph in memory.

3. Agentic Cognitive Frameworks

Autonomous agents require modular memory architectures to maintain state and execute long-horizon tasks without exceeding context window limitations.

The Memory Stack Architecture

- **Working Memory (State Management):** Managed via KV (Key-Value) caching in the inference engine. It retains the immediate context of the current execution loop and is flushed upon task completion.
- **Semantic Memory (Vector Retrieval):** Replaces internal model knowledge with external, verifiable data. Enterprises utilize Hierarchical Navigable Small World (HNSW) algorithms within vector databases (e.g., Milvus, Qdrant) to execute sub-millisecond similarity searches across millions of document chunks.

- **Episodic Memory (Event Logs):** Persistent storage of past user interactions and system decisions, typically stored in document databases (e.g., MongoDB). This enables the agent to recall context across independent sessions.
- **Procedural Memory (Execution Logic):** Encoded as Directed Acyclic Graphs (DAGs) or Finite State Machines (FSM). This defines the strict operational boundaries and API routing logic the agent must follow.

Adaptive Retrieval-Augmented Generation (RAG)

Static RAG simply prepends search results to a prompt. Adaptive RAG introduces routing algorithms. Before generating an answer, an evaluation layer determines if the retrieved context contains sufficient data. If gaps exist, the agent iteratively generates new search queries, synthesizing multiple disparate sources before passing the final context payload to the generation layer.

4. Multi-Agent Orchestration and Scaling

High-stakes environments cannot rely on a monolithic LLM. Multi-agent systems (MAS) distribute logic to specialized models, creating a peer-review architecture.

Consensus Protocols and Verification

To mitigate hallucinations, MAS employs Actor Model architectures where distinct agents handle generation, critique, and formatting.

- **Generator Agent:** Drafts the initial response or code block.
- **Critique Agent:** Evaluates the draft against ground-truth data or strict schema requirements. If verification fails, the payload is returned to the generator with specific error logs.

Mitigating Infrastructure Overhead

This peer-review loop drastically increases token consumption and API latency. Pragmatic scaling requires:

- **Model Routing:** Directing complex reasoning tasks to heavy models (e.g., GPT-4, Claude 3.5 Sonnet) while routing basic classification or formatting tasks to smaller, quantized local models (e.g., Llama 3 8B).
- **Semantic Caching:** Storing the vector embeddings of frequent queries. If a new query falls within a predefined similarity threshold of a cached query, the system returns the cached response, bypassing model inference entirely.

5. Enterprise Integration and Risk Management

Deploying agentic AI alters existing operational security and development pipelines.

Modernizing the SDLC

AI-assisted coding creates a bottleneck if the downstream pipeline cannot handle the increased code velocity. Agents must be integrated into the CI/CD pipeline to automate Application Security Testing (SAST/DAST), generate unit tests, and perform static analysis concurrently with code generation.

DevSecOps and Threat Modeling

As AI accelerates the automation of zero-day exploits, static firewall rules become insufficient. Security teams deploy defensive agentic workflows that continuously map network topologies, monitor anomaly vectors, and automatically isolate compromised endpoints before human intervention is required.

IT/OT Convergence for Asset Lifecycle

In Operational Technology (OT), agents ingest real-time telemetry from industrial IoT sensors. By combining this streaming data with semantic manuals and historical repair logs (Episodic Memory), agents issue deterministic work orders, route supply chain requests for replacement parts, and minimize unpredicted downtime.

6. Architecting Multi-Agent Systems for Scale and Low Latency

Multi-agent systems (MAS) inherently increase system latency due to sequential inference steps, API round-trips, and inter-agent communication overhead. Moving MAS from a proof-of-concept to enterprise production requires abandoning linear prompt chains in favor of tiered, asynchronous routing architectures.

Hierarchical Routing and Model Tiering

Executing all agentic logic through a flagship, high-parameter foundation model is computationally prohibitive and slow. Pragmatic scaling relies on a hierarchical topology driven by model tiering.

- **The Supervisor Agent (Router):** The entry point of the system is not a massive generative model, but a fast, heavily quantized local model (e.g., an INT8 8B parameter model). Its sole function is intent classification and payload delegation.
- **Specialized Workers:** The Supervisor routes sub-tasks to the appropriate tier. Basic data extraction is sent to a low-latency model, while complex synthesis or code generation is routed to a high-parameter reasoning engine. This ensures expensive compute is only utilized when strictly necessary.

Deterministic Execution via DAGs and FSMs

Unconstrained autonomous agents are prone to infinite loops and hallucinated reasoning cycles. Production multi-agent systems must constrain agent behavior using strict topological boundaries.

- **Directed Acyclic Graphs (DAGs):** Workflows are mapped as DAGs to enforce a forward-moving execution path. An agent cannot pass its output backward in the chain unless a specific, predefined error threshold is triggered during the critique phase.
- **Finite State Machines (FSM):** System states are hardcoded. An agent can only transition from "Data Retrieval" to "Draft Generation" if the payload meets a strict JSON schema validation. If the schema fails, the FSM forces an automatic retry loop with a hardcapped limit (e.g., maximum three retries) before throwing an exception to the user.

Latency Mitigation: The Scatter-Gather Pattern

Standard multi-agent loops operate sequentially, creating a severe bottleneck. To minimize Time-to-First-Byte (TTFB) and overall resolution time, the architecture must parallelize independent tasks.

When a complex query enters the system, the Supervisor utilizes a **Scatter-Gather** mechanism:

1. **Scatter:** The query is decomposed. Agent A queries an internal SQL database, Agent B initiates a web search for real-time market data, and Agent C searches the vector database for historical context—all simultaneously.
2. **Gather:** Asynchronous callbacks collect the data streams. A final Synthesizer agent aggregates the concurrent inputs into a single, cohesive output. This reduces the total system latency to the duration of the longest single sub-task, rather than the sum of all tasks.

Inter-Agent Semantic Caching

As agents communicate, they frequently request identical or highly similar data points. Implementing a semantic cache layer between agents drastically reduces inference overhead.

When Agent A queries Agent B, the request is embedded and checked against a fast, in-memory vector store (e.g., Redis with vector search). If the cosine similarity of the request exceeds a high confidence threshold (e.g., 0.98), the system serves the cached response instantly. This bypasses the need for Agent B to run an inference cycle, saving both compute resources and milliseconds of latency.